GlassQuill

Explainable Detection of AI-Generated Text

Project guide for the AUCML case study (TE7483) End Semester Evaluation

Course	Applications and Use Cases of Machine Learning (TE7483), 2 credits
Department	AIML Department, Symbiosis Institute of Technology, Pune
Batch and semester	2024-28 batch, Semester V
ESE weight	30 marks, 60% of the course
Prepared by	Rahul Narkhede, October 2026
Code repository	github.com/ThatOneRahul/glassquill (private, ask Rahul for an invite)

How to use this guide

Read sections 1 to 6 first. They explain the idea, the background, the data and the plan. Then follow section 7 step by step in Google Colab. Sections 8 and 9 build and deploy the web app. Section 10 shows how every part earns marks in the ESE rubric.

All code in this guide is in the repository. It was run end to end in October 2026 with the library versions in section 6, using a small dummy dataset and tiny stand-in models (the real data and models could not be downloaded on the test machine). Your real numbers will come from your own run on HC3. Section 14 lists the errors you are most likely to hit and how to fix them.

Contents

1. The project at a glance 4

1.1 Problem statement 4

1.2 Objective 4

1.3 Research questions 4

1.4 Stakeholders 4

1.5 Scope 4

1.6 Deliverables 4

2. Background you need 5

2.1 How an LLM writes 5

2.2 Perplexity 5

2.3 Burstiness 6

2.4 Stylometry 6

2.5 TF-IDF with logistic regression 6

2.6 XGBoost 6

2.7 DistilBERT 6

2.8 SHAP 6

2.9 LIME 6

2.10 Metrics and why false positives matter most 7

2.11 Data leakage and shortcuts 7

3. Datasets 7

3.1 Main dataset: HC3 7

3.2 Out-of-domain test set: MAGE 8

3.3 Paraphrased set 8

3.4 Final data plan 8

3.5 Optional: RAID benchmark 8

4. Methodology 8

4.1 Approach in one line 8

4.2 The four models 9

4.3 The 18 features used by XGBoost 9

4.4 Training strategy 9

4.5 Evaluation plan 10

4.6 Explanation plan 10

4.7 Limitations and ethics 10

5. Pipeline . . . 11

6. Technology stack . . . 12

7. Build it step by step . . . 12

7.1 Step 0: accounts . . . 12

7.2 Step 1: repository layout . . . 13

7.3 Step 2: open the project in Colab . . . 13

7.4 Step 3: load, clean and split the data . . . 13

7.5 Step 4: extract the features . . . 15

7.6 Step 5: train the three classic models . . . 16

7.7 Step 6: fine-tune DistilBERT . . . 17

7.8 Step 7: evaluate . . . 18

7.9 Step 8: explain . . . 19

7.10 Step 9: save the models and push . . . 22

8. The web app . . . 22

8.1 Try it in Colab first . . . 24

9. Deployment . . . 24

9.1 Plan A: Hugging Face Space on ZeroGPU . . . 24

9.2 Plan B: Colab share link (always works) . . . 25

9.3 Plan C: Streamlit Community Cloud . . . 25

9.4 Deployment checklist . . . 25

10. How this maps to the ESE rubric . . . 26

11. Plan and work split . . . 26

12. Report and presentation . . . 27

12.1 Report outline . . . 27

12.2 Future scope ideas . . . 27

12.3 Slide plan (12 slides) . . . 27

13. Viva preparation . . . 28

14. Troubleshooting . . . 29

15. References . . . 29

1. The project at a glance

1.1 Problem statement

Large language models such as ChatGPT, Gemini and Claude now write text that is very hard to tell apart from human writing. Colleges, news sites and hiring teams use AI detectors to flag such text. Most of these detectors give only a score with no reason. A wrong flag can hurt a student or a writer and they have no way to question it. This is a real problem: Liang et al. (2023) found that popular detectors flagged more than half of the essays written by non-native English speakers as AI-written.

1.2 Objective

Build a classifier that labels a piece of English text as **human-written** or **AI-generated** and also explains each decision in a way a teacher can understand. Then test how well it holds up on text from other LLMs and on paraphrased AI text.

1.3 Research questions

ID	Question
RQ1	Which simple signals (perplexity, sentence rhythm, vocabulary, punctuation) separate human and AI text and how strongly?
RQ2	How close can an explainable model (XGBoost with SHAP) get to a black-box model (fine-tuned DistilBERT)?
RQ3	How well do the detectors work on text from other LLMs and on paraphrased AI text?
RQ4	Do two explanation methods (SHAP and LIME) agree on why a text was flagged?

1.4 Stakeholders

Who	What they need from this project
Teachers and universities	A second opinion that comes with reasons, not a final verdict
Students and writers	A fair way to see and question why their text was flagged
Content platforms and newsrooms	Screening of large volumes of text with a clear audit trail
Researchers	A transparent baseline to compare new detectors against

1.5 Scope

- **In scope:** English text of 50 to 400 words, essay or question-answer style, two classes (human or AI).
- **Out of scope:** other languages, source code, documents that mix human and AI text, and naming which LLM wrote a text.

1.6 Deliverables

1. A cleaned dataset with a leak-free train, validation and test split
2. Four trained models and one comparison table
3. Robustness results on other LLMs (MAGE) and on paraphrased AI text
4. An explanation analysis: global SHAP, four local cases and two agreement checks

5. A web app with a public link that shows the verdict and the reasons
6. A GitHub repository, a final report and a slide deck

> **Why this counts as current research**
>
> Detecting AI text is an open problem. The RAID benchmark (2024) showed that most detectors break under simple attacks such as paraphrasing. Very few studies look at *why* a detector flags a text. This project combines a standard detection setup with two things that are still active research: explanations and robustness tests on unseen LLMs.

2. Background you need

You do not need to be an expert in all of this before you start. Use the table to plan your reading. The short notes after it cover the ideas you must be able to explain in the viva.

Topic	What to know	Where to learn	Time
Python, pandas, numpy	Load data, filter rows, groupby, apply	Kaggle Learn: Pandas	Known
ML basics	Train, validation and test sets, overfitting, cross-validation	scikit-learn user guide: Model selection	1 day
Classification metrics	Precision, recall, F1, ROC-AUC, confusion matrix	scikit-learn docs: Metrics	Half day
Text basics	Tokens, n-grams, TF-IDF	scikit-learn tutorial: Working with text data	Half day
Language models	Next-token prediction, logits, softmax, perplexity	Hugging Face LLM Course, chapter 1	1 day
Gradient boosting	Trees, learning rate, depth	XGBoost docs: Introduction to boosted trees	Half day
Fine-tuning transformers	Tokenizer, model, Trainer	Hugging Face LLM Course, chapter 3	1 day
Explainable AI	SHAP and LIME	SHAP docs intro; C. Molnar, Interpretable Machine Learning	1 day
Gradio	Blocks, inputs, outputs	Gradio quickstart	2 hours

2.1 How an LLM writes

An LLM writes one token (a word or part of a word) at a time. At each step it gives a probability to every possible next token and then picks one, usually from the most likely few. Because of this, AI text tends to use safe and expected words. Humans make more surprising choices: odd words, jokes, typos, sudden topic changes. Most of our features measure this difference.

2.2 Perplexity

Perplexity measures how surprised a language model is by a text. We use GPT-2 small as the "reader". For a text with N tokens:

$$\mathbf{PPL = \exp\left(-(1/N) \times \Sigma \log p(token_i \mid previous\ tokens) \right)}$$

Low perplexity means GPT-2 could guess the text easily, which is typical of AI text. "The cat sat on the mat" has low perplexity. "The cat negotiated with the mat" has high perplexity. We use the log of perplexity (the average surprise) as a feature. We also count how often the real token was GPT-2's

first guess or in its top ten guesses. This idea comes from the GLTR tool (Gehrmann et al., 2019).

2.3 Burstiness

Humans mix short and long sentences. AI text is often more even. We measure burstiness as the standard deviation of sentence length divided by the mean sentence length. A second version is the spread of token surprise inside the text (`nll_std`).

2.4 Stylometry

Stylometry means measuring writing style with numbers: average word length, vocabulary richness (unique words divided by total words), commas and other punctuation per word, common function words (the, of, and), first-person words (I, me, we), contractions (don't, it's) and readability scores.

2.5 TF-IDF with logistic regression

TF-IDF turns a text into a long vector with one weight per word or word pair. A word gets a high weight if it appears often in this text but rarely in others. Logistic regression then learns one coefficient per word. The contribution of a word to a prediction is simply coefficient × TF-IDF value, so the model is fully transparent. We call it the glass-box baseline.

2.6 XGBoost

XGBoost builds many small decision trees one after another. Each new tree tries to fix the mistakes of the trees before it. It works very well on tables of numeric features like ours, and SHAP can explain tree models exactly and quickly.

2.7 DistilBERT

DistilBERT is a smaller and faster version of BERT. According to its authors it is 40% smaller and 60% faster while keeping about 97% of BERT's language understanding. We fine-tune the whole network with a two-class output layer. It learns its own features from raw text, so it is usually the most accurate but also the hardest to explain.

2.8 SHAP

SHAP comes from game theory. Think of the features as players in a team and the prediction as the prize. The Shapley value of a feature is its average contribution to the prize over every possible order in which players could join. Key property: **base value + sum of all SHAP values = model output** (for XGBoost the output is in log-odds). TreeSHAP computes these values exactly for tree models. We use two views: global (beeswarm and bar plots over the whole test set) and local (a waterfall plot for one text).

2.9 LIME

LIME explains one prediction at a time. It removes random words from the text, watches how the model's output changes and fits a small linear model on these changes. The weights of that small model are the explanation. LIME works with any model but it is random and only approximate. That is why we check it against SHAP and against the exact word weights of the logistic regression.

2.10 Metrics and why false positives matter most

Metric	Meaning in this project (positive class = AI)
Precision	Of the texts we flagged as AI, how many really were AI
Recall	Of all AI texts, how many we caught
F1	Balance of precision and recall
ROC-AUC	How well the scores rank AI above human, for every possible threshold
False positive rate (FPR)	Share of human texts wrongly flagged as AI. This is the error that hurts real people, so we always report it next to F1

2.11 Data leakage and shortcuts

HC3 has human and AI answers to the *same* question. If one question appears in both train and test, the model can learn the topic instead of the writing style. So we split by question. Datasets also contain artifacts that give the answer away, for example the phrase "As an AI language model". A model that learns these gets near perfect scores that mean nothing. Rule of thumb: **if test accuracy is above 99%, look for a shortcut before you celebrate.**

3. Datasets

3.1 Main dataset: HC3

Item	Details
Name	HC3: Human ChatGPT Comparison Corpus (Guo et al., 2023)
Hub id	`Hello-SimpleAI/HC3` on Hugging Face
Content	Questions with human answers and ChatGPT answers from five sources: reddit_eli5, open_qa, wiki_csai, finance and medicine
Size (English)	About 24 thousand questions with about 58 thousand human and 27 thousand ChatGPT answers. The `all.jsonl` file is 73.7 MB
Fields	`question`, `human_answers` (list), `chatgpt_answers` (list), `source`
License	CC BY-SA 4.0 (stricter licences of the original sources still apply)

What we found when we checked the HC3 files

1. ChatGPT answers store line breaks as the literal text `\n`. Human answers do not. Left in, the model learns "backslash n means AI".
2. Some human answers from Wikipedia and ELI5 have a space before commas and full stops, like "word , word". This is another shortcut.
3. Some ChatGPT answers contain giveaway phrases such as "As an AI language model".
4. The dataset repo still uses an old loading script (`HC3.py`). The `datasets` library dropped support for loading scripts in version 4.0, so `load_dataset("Hello-SimpleAI/HC3")` now fails. Our `load_hc3()` reads `all.jsonl` directly with pandas instead.

`clean_text()` in `src/data.py` fixes points 1 to 3.

3.2 Out-of-domain test set: MAGE

To see if the detector works beyond ChatGPT on HC3 topics, we test it on MAGE (Li et al., 2024). Its test split has 60.7 thousand texts from many domains written by humans and by many different LLMs. The `src` column says where each text came from. Licence: Apache 2.0. We load the test split as a single 45 MB parquet file and sample 1,000 texts per class with 50 to 400 words.

> **Two traps in MAGE**
>
> - **The labels are flipped.** MAGE uses 1 = human and 0 = machine. We use 1 = AI. Always do `ood["label"] = 1 - ood["label"]`. If you forget, ROC-AUC on MAGE comes out below 0.5.
> - **Reddit footer.** Human texts from the ChangeMyView subreddit end with a moderator note starting "Hello, users of CMV!". It is a free hint for the model, so we cut it off.

3.3 Paraphrased set

We take 150 AI texts from our own test set and rewrite every sentence with a T5 paraphrasing model (`humarin/chatgpt_paraphraser_on_T5_base`). Then we check how many are still caught. This copies what a student might do to hide AI use.

3.4 Final data plan

Set	Source	Human	AI	Used for
Train	HC3	about 4,200	about 4,200	Fitting models and 5-fold grouped cross-validation
Validation	HC3	about 900	about 900	Perplexity threshold and choosing the best DistilBERT epoch
Test	HC3	about 900	about 900	Main in-domain results. Used once at the end
Out-of-domain	MAGE test	1,000	1,000	Other LLMs and other domains
Paraphrased	HC3 test	0	150	Robustness to paraphrasing

Counts in the first three rows vary a little because the split is done by question, not by row.

3.5 Optional: RAID benchmark

RAID (Dugan et al., 2024) is the largest benchmark for AI text detectors: 11 LLMs, 11 genres and 12 adversarial attacks. Its training file alone is 11.8 GB, so we only mention it as future work.

4. Methodology

4.1 Approach in one line

Compare four models that go from simple and transparent to powerful and opaque, test them in three ways and explain them in three ways.

4.2 The four models

#	Model	Input	Why we include it	How we explain it
1	Perplexity threshold	GPT-2 log perplexity	Zero-shot baseline. Shows how far one signal goes	The number itself
2	TF-IDF + logistic regression	Word and word-pair weights	Classic glass-box text model	Exact word contributions
3	XGBoost	18 style and perplexity features	Main model. Strong on tables and its features mean something	TreeSHAP, global and local
4	DistilBERT	Raw text, up to 256 tokens	Strong black box to compare against	LIME and SHAP text

4.3 The 18 features used by XGBoost

The last column is what we *expect*. SHAP will show whether the data agrees. That makes a good discussion point in the report.

Feature	What it measures	Expected for AI text
`log_ppl`	Average surprise of GPT-2 (log perplexity)	Lower
`nll_std`	How uneven the surprise is across tokens	Lower
`nll_max`	Surprise of the most surprising token	Lower
`top1_frac`	Share of tokens that were GPT-2's first guess	Higher
`top10_frac`	Share of tokens in GPT-2's top ten guesses	Higher
`mean_entropy`	How unsure GPT-2 is about the next token	Lower
`avg_sent_len`	Words per sentence	About the same or higher
`std_sent_len`, `burstiness`	Variation in sentence length	Lower
`type_token_ratio`	Unique words divided by total words	Unclear, check
`avg_word_len`	Letters per word	Higher
`comma_rate`, `punct_rate`	Commas and all punctuation per word	Unclear, check
`function_word_rate`	Share of words like the, of, and	Unclear, check
`first_person_rate`	Share of I, me, my, we, us, our	Lower
`contraction_rate`	Share of words like don't, it's	Lower
`flesch_reading_ease`, `fk_grade`	Readability	Harder to read (lower ease, higher grade)

4.4 Training strategy

- Fixed random seed 42 everywhere so results can be repeated.
- Tuning uses 5-fold **GroupKFold** on the training set only, grouped by question, so the folds are also leak-free.
- Logistic regression: C in {0.1, 1, 10}. XGBoost: 400 trees, depth in {3, 4, 6}, learning rate in {0.05, 0.1}. Both are tuned for F1.
- Perplexity threshold: chosen on the validation set with Youden's J, the point on the ROC curve where true positive rate minus false positive rate is highest.

- DistilBERT: learning rate 2e-5, batch size 16, 2 epochs, 256 tokens, keep the epoch with the best validation F1.
- The test set is used only once, after every choice is final.

4.5 Evaluation plan

- **Metrics:** accuracy, precision, recall, F1, ROC-AUC, false positive rate, confusion matrix and ROC curve.
- **Three test conditions:** in-domain (HC3 test), out-of-domain (MAGE) and paraphrased AI text.
- **Error analysis:** per-source accuracy on MAGE to see which LLMs fool the model most.

Hypothesis	How we test it
H1: AI text has lower perplexity than human text	Feature means per class and SHAP direction of `log_ppl`
H2: AI text has lower burstiness	Same check for `burstiness` and `nll_std`
H3: DistilBERT wins in-domain but drops the most out-of-domain	Compare F1 on HC3 test and MAGE
H4: Paraphrasing lowers recall for every model	Recall before and after paraphrasing

4.6 Explanation plan

- **Global:** SHAP beeswarm and bar plots for XGBoost over the whole test set.
- **Local:** SHAP waterfall plots and plain-language reasons for four texts: a correct AI, a correct human, one false positive and one false negative.
- **Glass box:** top words of the TF-IDF model, overall and for single texts.
- **Black box:** LIME and SHAP text explanations for DistilBERT.
- **Check 1:** overlap of the top ten words from SHAP and LIME for DistilBERT (Jaccard score).
- **Check 2:** how many of the true top ten words of the logistic regression LIME can find. This tells us how much to trust LIME.

4.7 Limitations and ethics

- A detector score must never be the only evidence against a person.
- Detectors can be unfair to non-native writers, whose simpler wording looks "predictable".
- Paraphrasing tools and newer LLMs can make the model fail. Results can go stale quickly.
- We only cover English and only the domains in HC3 and MAGE.

5. Pipeline

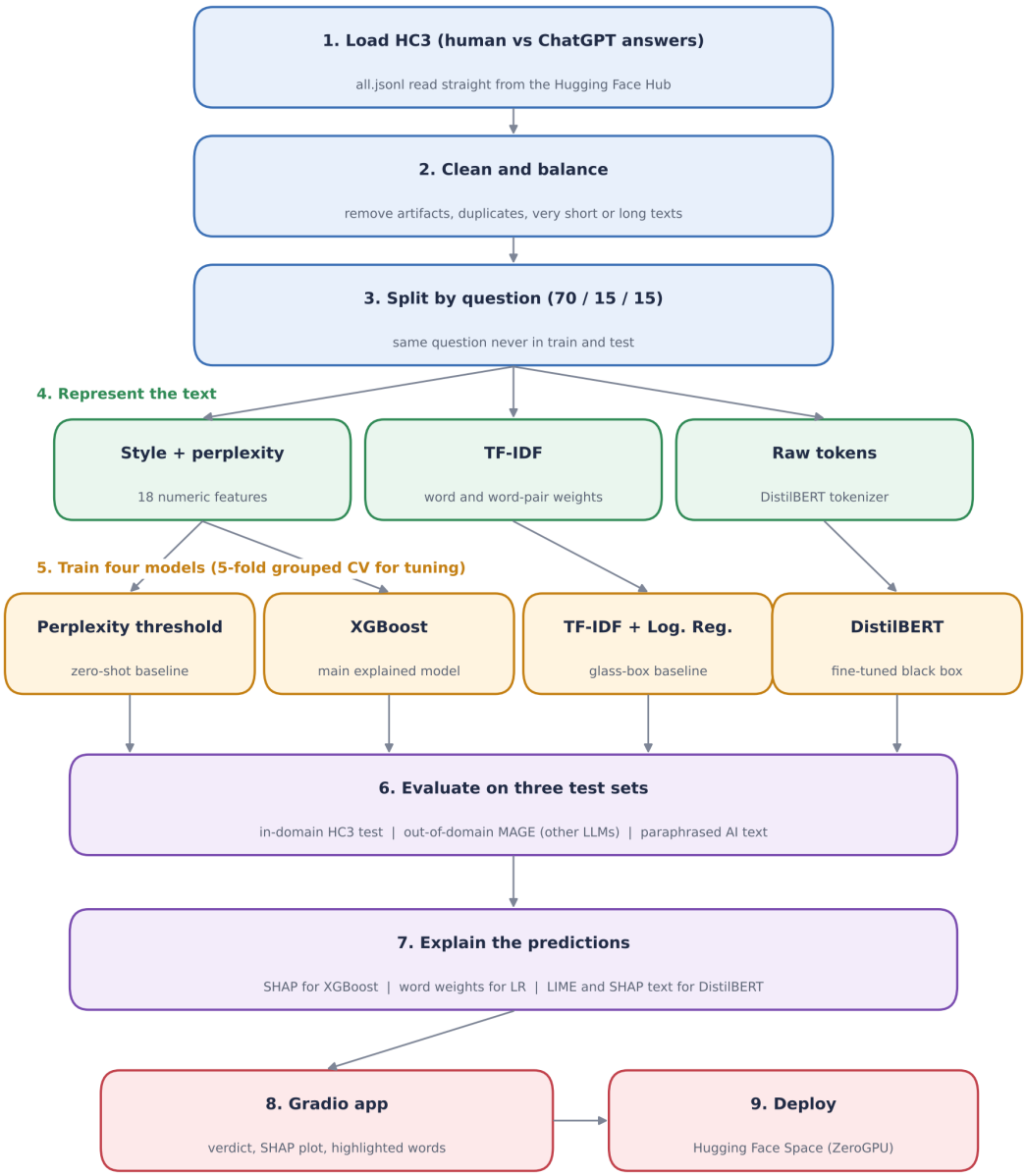


Figure 1. The full GlassQuill pipeline from raw data to the deployed app.

Steps 1 to 3 are in `src/data.py`, step 4 in `src/features.py`, step 5 in `src/models.py` and `src/train_distilbert.py`, step 7 in `src/explain.py` and steps 8 and 9 in `app.py`. The notebook cells in section 7 call these files in order.

6. Technology stack

Layer	Tool	Version tested	Used for
Compute	Google Colab with a free T4 GPU	-	Feature extraction and training
Language	Python	3.12 on Colab	Everything
Data	pandas, numpy, pyarrow	2.2 or 3.0, 2.x, recent	Tables and parquet files
Classic ML	scikit-learn	1.9.1	TF-IDF, logistic regression, CV, metrics
Boosting	xgboost	3.4.1	Main model
Deep learning	torch, transformers, accelerate, datasets	2.14, 5.20.2, 1.15, 5.1.0	GPT-2 scores and DistilBERT
Text features	textstat, nltk	0.7.13, 3.10	Readability scores
Explainability	shap, lime	0.54.1, 0.2.0.1	SHAP and LIME
Plots	matplotlib	3.11	Figures for the report
Web app	gradio (streamlit as plan C)	6.30.0 (1.66)	Demo interface
Hosting	Hugging Face Spaces (ZeroGPU), hf command line tool	huggingface_hub 2.3	Public demo link
Code	Git and GitHub	-	Version control and sharing

Newer versions should also work. If something breaks, pin the versions in this table in `requirements.txt`.

7. Build it step by step

7.1 Step 0: accounts

- **Google account** for Colab.
- **GitHub account.** Rahul will invite you to the private repo `github.com/ThatOneRahul/glassquill`. Accept the invite at `github.com/ThatOneRahul/glassquill/invitations`.
- **Hugging Face account, today.** Free hosting on ZeroGPU needs an account that is older than 30 days with a verified email. If the account is newer than that on demo day, use plan B in section 9.

7.2 Step 1: repository layout

Folder structure

```
glassquill/
├── app.py                Gradio app, also the Space entry point
├── streamlit_app.py      plan C front end (optional)
├── requirements.txt
├── README.md             project readme
├── notebook_cells.py     all notebook cells in order (copy into Colab)
├── docs/
│   └── GlassQuill_Project_Guide.pdf
├── src/
│   ├── __init__.py       empty file
│   ├── data.py           load, clean, balance, split
│   ├── features.py       style and perplexity features
│   ├── models.py         baseline, logistic regression, XGBoost, evaluate()
│   ├── train_distilbert.py
│   ├── explain.py        SHAP, LIME, word weights, highlights
│   └── detector.py       loads saved models and analyses one text
├── models/               saved models (created in step 9)
├── data/                 parquet splits (not committed)
└── reports/figures/      plots for the report
```

7.3 Step 2: open the project in Colab

In Colab choose **Runtime › Change runtime type › T4 GPU**. Because the repo is private, Colab needs a GitHub token to clone it. On GitHub go to **Settings › Developer settings › Personal access tokens › Fine-grained tokens**, create a token with access to the glassquill repo and **Contents: Read and write** permission. In Colab click the key icon on the left (Secrets), add it as `GITHUB_TOKEN` and allow notebook access. Never paste the token into a cell.

Notebook cell: clone and install

```
from google.colab import userdata
token = userdata.get("GITHUB_TOKEN")
!git clone https://{token}@github.com/ThatOneRahul/glassquill.git
%cd glassquill
!pip install -q -r requirements.txt
```

Then run the setup cell. It fixes the random seeds and makes the output folders.

Notebook cell: setup

```
import os, json, random
import numpy as np, pandas as pd, torch
import matplotlib.pyplot as plt

SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
dev = "cuda" if torch.cuda.is_available() else "cpu"
for d in ["data", "models", "reports/figures"]:
    os.makedirs(d, exist_ok=True)
print("device:", dev)
```

7.4 Step 3: load, clean and split the data

src/data.py

```
"""Load HC3, clean the text, balance classes and make a leak-free split."""
import re
import pandas as pd
from sklearn.model_selection import GroupShuffleSplit

HC3_URL = "https://huggingface.co/datasets/Hello-SimpleAI/HC3/resolve/main/all.jsonl"

# Phrases that give the answer away. If we keep them the model learns a shortcut.
GIVEAWAYS = [
    r"as an ai language model[^.]*\.?",
```

```
    r"i'?m sorry,? but i[^.]*\.?",
    r"i am not able to[^.]*\.?",
    r"i hope this helps[^.]*\.?",
]


def load_hc3(url=HC3_URL):
    """One row per answer: qid, source, text, label (0 = human, 1 = AI)."""
    raw = pd.read_json(url, lines=True)
    rows = []
    for qid, r in raw.iterrows():
        for t in r["human_answers"]:
            rows.append({"qid": qid, "source": r["source"], "text": t, "label": 0})
        for t in r["chatgpt_answers"]:
            rows.append({"qid": qid, "source": r["source"], "text": t, "label": 1})
    return pd.DataFrame(rows)


def clean_text(t):
    t = str(t).replace("\\n", " ").replace("\n", " ")   # HC3 stores literal "\n"
    t = re.sub(r"\s+([,.!?;:])", r"\1", t)              # "word ," -> "word,"
    for p in GIVEAWAYS:
        t = re.sub(p, "", t, flags=re.IGNORECASE)
    return re.sub(r"\s+", " ", t).strip()


def prepare(df, min_words=50, max_words=400, per_class=6000, seed=42):
    df = df.dropna(subset=["text"]).copy()
    df["text"] = df["text"].map(clean_text)
    df = df.drop_duplicates("text")
    df["n_words"] = df["text"].str.split().str.len()
    df = df[df["n_words"].between(min_words, max_words)]
    n = min(per_class, df["label"].value_counts().min())   # same count per class
    df = df.groupby("label").sample(n=n, random_state=seed)
    return df.reset_index(drop=True)


def group_split(df, seed=42):
    """70/15/15 split where one question never appears in two splits."""
    gss = GroupShuffleSplit(n_splits=1, test_size=0.30, random_state=seed)
    tr, rest = next(gss.split(df, groups=df["qid"]))
    train, rest = df.iloc[tr], df.iloc[rest]
    gss2 = GroupShuffleSplit(n_splits=1, test_size=0.50, random_state=seed)
    va, te = next(gss2.split(rest, groups=rest["qid"]))
    val, test = rest.iloc[va], rest.iloc[te]
    assert not set(train.qid) & set(test.qid), "question leak between train and test"
    return [d.reset_index(drop=True) for d in (train, val, test)]
```

Notebook cell: data

```
from src.data import load_hc3, prepare, group_split

raw = load_hc3()
print(raw.shape, raw["label"].value_counts().to_dict())
df = prepare(raw, per_class=6000)
train, val, test = group_split(df, seed=SEED)
for name, d in [("train", train), ("val", val), ("test", test)]:
    d.to_parquet(f"data/{name}.parquet")
    print(name, len(d), "share AI:", round(d["label"].mean(), 3))
```

Now look at the data before training anything. Check how many texts each source has and compare the length histograms of the two classes. If AI texts are all much longer, length becomes a shortcut. Then narrow the word window in `prepare()`.

Notebook cell: eda

```
print(df.groupby(["source", "label"]).size().unstack())       # texts per domain
fig, ax = plt.subplots(figsize=(6, 4))
for lab, name in [(0, "Human"), (1, "AI")]:
    ax.hist(df.loc[df["label"] == lab, "n_words"], bins=40, alpha=0.6, label=name)
ax.set_xlabel("words per text"); ax.legend()
fig.savefig("reports/figures/length_hist.png", dpi=200)
```

7.5 Step 4: extract the features

src/features.py

```python
"""Hand-made features: writing style (stylometry) and GPT-2 perplexity."""
import re
import string
import numpy as np
import pandas as pd
import nltk
import textstat
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

try:                                    # textstat needs this word list for syllables
    nltk.data.find("corpora/cmudict")
except LookupError:
    nltk.download("cmudict", quiet=True)

SENT_SPLIT = re.compile(r"(?<=[.!?])\s+")
WORD_RE = re.compile(r"[A-Za-z']+")
FUNCTION_WORDS = {"the", "of", "and", "to", "a", "in", "that", "is", "it", "for",
                  "as", "with", "be", "on", "not", "this", "but", "by", "or", "which"}
FIRST_PERSON = {"i", "me", "my", "mine", "we", "us", "our"}


def stylometric_features(text):
    sents = [s for s in SENT_SPLIT.split(text) if s.strip()]
    words = WORD_RE.findall(text.lower())
    n = max(len(words), 1)
    lens = np.array([len(WORD_RE.findall(s)) for s in sents] or [n])
    return {
        "avg_sent_len": lens.mean(),
        "std_sent_len": lens.std(),
        "burstiness": lens.std() / (lens.mean() + 1e-9),   # spread of sentence length
        "type_token_ratio": len(set(words)) / n,           # vocabulary richness
        "avg_word_len": np.mean([len(w) for w in words]) if words else 0.0,
        "comma_rate": text.count(",") / n,
        "punct_rate": sum(c in string.punctuation for c in text) / n,
        "function_word_rate": sum(w in FUNCTION_WORDS for w in words) / n,
        "first_person_rate": sum(w in FIRST_PERSON for w in words) / n,
        "contraction_rate": sum("'" in w for w in words) / n,
        "flesch_reading_ease": textstat.flesch_reading_ease(text),
        "fk_grade": textstat.flesch_kincaid_grade(text),
    }


class PerplexityScorer:
    """How surprised GPT-2 is by each token. AI text is usually less surprising."""

    def __init__(self, model_name="gpt2", device=None, max_tokens=256):
        self.device = device or ("cuda" if torch.cuda.is_available() else "cpu")
        self.tok = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModelForCausalLM.from_pretrained(model_name).to(self.device).eval()
        self.max_tokens = max_tokens

    @torch.no_grad()
    def token_scores(self, text):
        enc = self.tok(text, return_tensors="pt", truncation=True,
                       max_length=self.max_tokens).to(self.device)
        ids = enc["input_ids"]
        logp = torch.log_softmax(self.model(**enc).logits[0, :-1].float(), dim=-1)
        target = ids[0, 1:]
        tok_lp = logp.gather(-1, target.unsqueeze(-1)).squeeze(-1)
        ranks = (logp > tok_lp.unsqueeze(-1)).sum(-1) + 1      # 1 = model's top guess
        entropy = -(logp.exp() * logp).sum(-1)
        tokens = self.tok.convert_ids_to_tokens(target.tolist())
        return tokens, (-tok_lp).cpu().numpy(), ranks.cpu().numpy(), entropy.cpu().numpy()

    def features(self, text):
        _, nll, ranks, ent = self.token_scores(text)
        return {
            "log_ppl": float(nll.mean()),          # log perplexity
            "nll_std": float(nll.std()),           # how uneven the surprise is
```

```
            "nll_max": float(nll.max()),
            "top1_frac": float((ranks == 1).mean()),
            "top10_frac": float((ranks <= 10).mean()),
            "mean_entropy": float(ent.mean()),
        }


def build_feature_table(texts, scorer):
    rows = []
    for t in texts:
        f = stylometric_features(t)
        f.update(scorer.features(t))
        rows.append(f)
    return pd.DataFrame(rows)
```

This takes a few minutes on a T4 GPU. The results are saved as parquet files, so if Colab disconnects you can load them again instead of starting over. The printed table shows the mean of each feature per class. Write down which features differ the most. That is your first result for RQ1.

Notebook cell: features

```
from src.features import PerplexityScorer, build_feature_table

scorer = PerplexityScorer("gpt2")                # uses the Colab GPU if present
X = {}
for name, d in [("train", train), ("val", val), ("test", test)]:
    X[name] = build_feature_table(d["text"], scorer)
    X[name].to_parquet(f"data/X_{name}.parquet")
FEATS = list(X["train"].columns)
print(X["train"].assign(label=train["label"]).groupby("label").mean().T.round(3))
```

7.6 Step 5: train the three classic models

src/models.py

```
"""Train the three classic models and score them with the same metrics."""
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, confusion_matrix, f1_score,
                             precision_score, recall_score, roc_auc_score, roc_curve)
from sklearn.model_selection import GridSearchCV, GroupKFold
from sklearn.pipeline import make_pipeline
from xgboost import XGBClassifier


def evaluate(name, y_true, score, threshold=0.5):
    pred = (np.asarray(score) >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, pred, labels=[0, 1]).ravel()
    return {"model": name,
            "accuracy": accuracy_score(y_true, pred),
            "precision": precision_score(y_true, pred, zero_division=0),
            "recall": recall_score(y_true, pred),
            "f1": f1_score(y_true, pred),
            "roc_auc": roc_auc_score(y_true, score),
            "fpr": fp / (fp + tn)}          # humans wrongly flagged as AI


# Model 1: zero-shot perplexity threshold (no training at all)
def perplexity_threshold(val_logppl, y_val):
    """Lower perplexity = more AI-like, so the score is -log_ppl.
    Pick the cut-off on the validation set with Youden's J."""
    fpr, tpr, thr = roc_curve(y_val, -val_logppl)
    return thr[np.argmax(tpr - fpr)]


# Model 2: TF-IDF + logistic regression (interpretable by design)
def train_tfidf_lr(train, seed=42):
    pipe = make_pipeline(
        TfidfVectorizer(ngram_range=(1, 2), min_df=3, max_features=50000, sublinear_tf=True),
        LogisticRegression(max_iter=2000, random_state=seed))
```

```
    grid = GridSearchCV(pipe, {"logisticregression__C": [0.1, 1, 10]},
                        cv=GroupKFold(n_splits=5), scoring="f1", n_jobs=-1)
    grid.fit(train["text"], train["label"], groups=train["qid"])
    print("best C:", grid.best_params_)
    return grid.best_estimator_


# Model 3: XGBoost on style + perplexity features (explained with SHAP)
def train_xgb(X_train, y_train, groups, seed=42):
    base = XGBClassifier(n_estimators=400, subsample=0.8, colsample_bytree=0.8,
                         eval_metric="logloss", random_state=seed)
    grid = GridSearchCV(base, {"max_depth": [3, 4, 6], "learning_rate": [0.05, 0.1]},
                        cv=GroupKFold(n_splits=5), scoring="f1", n_jobs=-1)
    grid.fit(X_train, y_train, groups=groups)
    print("best params:", grid.best_params_)
    return grid.best_estimator_
```

Notebook cell: train

```
from src.models import evaluate, perplexity_threshold, train_tfidf_lr, train_xgb

thr = perplexity_threshold(X["val"]["log_ppl"].values, val["label"].values)
lr = train_tfidf_lr(train, seed=SEED)
xgb = train_xgb(X["train"][FEATS], train["label"], groups=train["qid"], seed=SEED)
```

7.7 Step 6: fine-tune DistilBERT

src/train_distilbert.py

```
"""Model 4: fine-tune DistilBERT (most accurate, but a black box)."""
import numpy as np
import torch
from datasets import Dataset
from sklearn.metrics import accuracy_score, f1_score
from transformers import (AutoModelForSequenceClassification, AutoTokenizer,
                          DataCollatorWithPadding, Trainer, TrainingArguments)

MODEL = "distilbert-base-uncased"


def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return {"accuracy": accuracy_score(labels, preds), "f1": f1_score(labels, preds)}


def train_distilbert(train, val, out_dir="models/distilbert", model_name=MODEL,
                     epochs=2, seed=42):
    tok = AutoTokenizer.from_pretrained(model_name)

    def to_ds(df):
        ds = Dataset.from_pandas(df[["text", "label"]], preserve_index=False)
        return ds.map(lambda b: tok(b["text"], truncation=True, max_length=256), batched=True)

    model = AutoModelForSequenceClassification.from_pretrained(
        model_name, num_labels=2,
        id2label={0: "Human", 1: "AI"}, label2id={"Human": 0, "AI": 1})
    args = TrainingArguments(
        output_dir=out_dir, eval_strategy="epoch", save_strategy="epoch",
        learning_rate=2e-5, per_device_train_batch_size=16, per_device_eval_batch_size=32,
        num_train_epochs=epochs, weight_decay=0.01, load_best_model_at_end=True,
        metric_for_best_model="f1", save_total_limit=1,
        fp16=torch.cuda.is_available(), report_to="none", seed=seed)
    trainer = Trainer(model=model, args=args, train_dataset=to_ds(train),
                      eval_dataset=to_ds(val), processing_class=tok,
                      data_collator=DataCollatorWithPadding(tok),
                      compute_metrics=compute_metrics)
    trainer.train()
    trainer.save_model(out_dir)
    tok.save_pretrained(out_dir)
    return trainer
```

```
@torch.no_grad()
def predict_proba(texts, model, tok, device="cpu", batch_size=32):
    """Returns an (n, 2) array of [P(human), P(AI)]. Also used by LIME."""
    model.to(device).eval()
    out = []
    for i in range(0, len(texts), batch_size):
        enc = tok(list(texts[i:i + batch_size]), truncation=True, max_length=256,
                  padding=True, return_tensors="pt").to(device)
        out.append(torch.softmax(model(**enc).logits, dim=-1).cpu().numpy())
    return np.vstack(out)
```

Two epochs take roughly ten minutes on a T4. If you run out of GPU memory, lower the batch size to 8.

Notebook cell: bert

```
from transformers import AutoModelForSequenceClassification, AutoTokenizer
from src.train_distilbert import train_distilbert, predict_proba

train_distilbert(train, val, out_dir="models/distilbert", seed=SEED)
bert = AutoModelForSequenceClassification.from_pretrained("models/distilbert")
bert_tok = AutoTokenizer.from_pretrained("models/distilbert")
```

7.8 Step 7: evaluate

`score_all()` gives every model a score where higher means more AI-like, plus the threshold to use. The same function is reused for MAGE and the paraphrased set, so all numbers are comparable.

Notebook cell: evaluate

```
def score_all(d, Xd):
    """Score and threshold for every model. Higher score = more AI-like."""
    return {
        "Perplexity threshold": (-Xd["log_ppl"].values, thr),
        "TF-IDF + LR": (lr.predict_proba(d["text"])[:, 1], 0.5),
        "XGBoost": (xgb.predict_proba(Xd[FEATS])[:, 1], 0.5),
        "DistilBERT": (predict_proba(list(d["text"]), bert, bert_tok, device=dev)[:, 1], 0.5),
    }

scores_test = score_all(test, X["test"])
results = pd.DataFrame([evaluate(m, test["label"], s, t) for m, (s, t) in scores_test.items()])
results.round(3).to_csv("reports/results_in_domain.csv", index=False)
print(results.round(3).to_string(index=False))
```

Notebook cell: plots

```
from sklearn.metrics import ConfusionMatrixDisplay, RocCurveDisplay

fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, (m, (s, t)) in zip(axes, scores_test.items()):
    ConfusionMatrixDisplay.from_predictions(
        test["label"], (s >= t).astype(int), display_labels=["Human", "AI"], ax=ax, colorbar=False)
    ax.set_title(m)
plt.tight_layout(); fig.savefig("reports/figures/confusion_matrices.png", dpi=200)

fig, ax = plt.subplots(figsize=(6, 5))
for m, (s, t) in scores_test.items():
    RocCurveDisplay.from_predictions(test["label"], s, name=m, ax=ax)
fig.savefig("reports/figures/roc_curves.png", dpi=200)
```

Out-of-domain test on MAGE. Note the label flip and the footer removal.

Notebook cell: ood

```
from src.data import clean_text

MAGE = ("https://huggingface.co/datasets/yaful/MAGE/resolve/"
        "refs%2Fconvert%2Fparquet/default/test/0000.parquet")
ood = pd.read_parquet(MAGE)
ood["label"] = 1 - ood["label"]                  # MAGE uses 1 = human. We use 1 = AI
ood["text"] = (ood["text"].str.replace(r"(?s)Hello, users of CMV!.*$", "", regex=True)
               .map(clean_text))                 # drop the Reddit moderator footer
ood["n_words"] = ood["text"].str.split().str.len()
ood = (ood[ood["n_words"].between(50, 400)]
       .groupby("label").sample(n=1000, random_state=SEED).reset_index(drop=True))

X_ood = build_feature_table(ood["text"], scorer)
scores_ood = score_all(ood, X_ood)
ood_results = pd.DataFrame([evaluate(m, ood["label"], s, t)
                            for m, (s, t) in scores_ood.items()])
ood_results.round(3).to_csv("reports/results_out_of_domain.csv", index=False)
print(ood_results.round(3).to_string(index=False))

# which generators fool the main model most?
correct = (scores_ood["XGBoost"][0] >= 0.5).astype(int) == ood["label"]
print(ood.assign(correct=correct).groupby("src")["correct"].agg(["mean", "size"])
      .sort_values("mean").head(10))
```

Paraphrase test. Expect this cell to take several minutes because every sentence is rewritten.

Notebook cell: paraphrase

```
from transformers import AutoModelForSeq2SeqLM
from src.features import SENT_SPLIT

PP = "humarin/chatgpt_paraphraser_on_T5_base"
pp_tok = AutoTokenizer.from_pretrained(PP)
pp = AutoModelForSeq2SeqLM.from_pretrained(PP).to(dev)

def paraphrase(text):
    out = []
    for s in SENT_SPLIT.split(text):
        enc = pp_tok("paraphrase: " + s, return_tensors="pt", truncation=True,
                     max_length=128).to(dev)
        ids = pp.generate(**enc, num_beams=4, max_length=128, no_repeat_ngram_size=3)
        out.append(pp_tok.decode(ids[0], skip_special_tokens=True))
    return " ".join(out)

ai_test = test[test["label"] == 1].sample(n=150, random_state=SEED).reset_index(drop=True)
para = ai_test.assign(text=ai_test["text"].map(paraphrase))
before = score_all(ai_test, build_feature_table(ai_test["text"], scorer))
after = score_all(para, build_feature_table(para["text"], scorer))
robust = pd.DataFrame({m: {"recall_before": (before[m][0] >= t).mean(),
                           "recall_after": (after[m][0] >= t).mean()}
                       for m, (_, t) in before.items()}).T
print(robust.round(3))
```

7.9 Step 8: explain

src/explain.py

```
"""Explanations: SHAP for XGBoost, word weights for TF-IDF LR, LIME for DistilBERT."""
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import shap
from lime.lime_text import LimeTextExplainer

NICE_NAMES = {
    "log_ppl": "Perplexity (log)", "nll_std": "Unevenness of surprise",
    "nll_max": "Most surprising token", "top1_frac": "Share of top-1 guessable words",
    "top10_frac": "Share of top-10 guessable words", "mean_entropy": "GPT-2 uncertainty",
```

```
    "avg_sent_len": "Average sentence length", "std_sent_len": "Sentence length spread",
    "burstiness": "Burstiness", "type_token_ratio": "Vocabulary richness",
    "avg_word_len": "Average word length", "comma_rate": "Commas per word",
    "punct_rate": "Punctuation per word", "function_word_rate": "Function word rate",
    "first_person_rate": "First-person words", "contraction_rate": "Contractions",
    "flesch_reading_ease": "Reading ease", "fk_grade": "Grade level",
}


def shap_global(model, X, out_path="reports/figures/shap_beeswarm.png"):
    sv = shap.TreeExplainer(model)(X)
    shap.plots.beeswarm(sv, max_display=15, show=False)
    plt.tight_layout(); plt.savefig(out_path, dpi=200); plt.close()
    return sv


def shap_local_figure(explainer, x_row):
    """Waterfall plot for one text. x_row is a one-row DataFrame."""
    sv = explainer(x_row)
    shap.plots.waterfall(sv[0], max_display=10, show=False)
    fig = plt.gcf(); fig.set_size_inches(8, 5); plt.tight_layout()
    return fig, sv[0]


def plain_reasons(sv_row, top_k=3):
    """Turn the biggest SHAP values into short sentences for the user."""
    order = np.argsort(-np.abs(sv_row.values))[:top_k]
    lines = []
    for i in order:
        name = sv_row.feature_names[i]
        side = "AI" if sv_row.values[i] > 0 else "human"
        lines.append(f"- {NICE_NAMES.get(name, name)} = {sv_row.data[i]:.2f} "
                     f"pushed the score towards {side}")
    return "\n".join(lines)


def lr_word_weights(pipe, text):
    """Contribution of each word in the text for the TF-IDF + LR model."""
    vec = pipe.named_steps["tfidfvectorizer"]
    lr = pipe.named_steps["logisticregression"]
    x = vec.transform([text]).multiply(lr.coef_[0]).tocsr()
    vocab = vec.get_feature_names_out()
    return {vocab[j]: v for j, v in zip(x.indices, x.data)}


def highlight(text, weights):
    """(word, score) pairs for gr.HighlightedText. Positive = AI-like."""
    top = max((abs(v) for v in weights.values()), default=1.0) or 1.0
    out = []
    for w in text.split():
        key = w.lower().strip(".,!?;:\"'()")
        out.append((w + " ", float(weights[key] / top) if key in weights else None))
    return out


def lime_explain(predict_proba, text, num_features=10, num_samples=500):
    explainer = LimeTextExplainer(class_names=["Human", "AI"], random_state=42)
    exp = explainer.explain_instance(text, predict_proba, num_features=num_features,
                                     num_samples=num_samples)
    return exp.as_list()   # [(word, weight), ...]
```

Notebook cell: shap

```
import shap
from src.explain import shap_global, shap_local_figure, plain_reasons

sv = shap_global(xgb, X["test"][FEATS])               # saves the beeswarm plot
shap.plots.bar(sv, max_display=15, show=False)
plt.tight_layout(); plt.savefig("reports/figures/shap_bar.png", dpi=200); plt.close()

p = scores_test["XGBoost"][0]; y = test["label"].values; pred = (p >= 0.5).astype(int)
def first(mask):
    idx = np.where(mask)[0]
    return int(idx[0]) if len(idx) else None

cases = {"true_ai": first((y == 1) & (pred == 1)),
         "true_human": first((y == 0) & (pred == 0)),
         "false_positive": first((y == 0) & (pred == 1)),
         "false_negative": first((y == 1) & (pred == 0))}
explainer = shap.TreeExplainer(xgb)
for name, i in cases.items():
    if i is None:
        continue
    fig, row = shap_local_figure(explainer, X["test"][FEATS].iloc[[i]])
    fig.savefig(f"reports/figures/waterfall_{name}.png", dpi=200); plt.close(fig)
    print(f"--- {name}  P(AI) = {p[i]:.2f}\n{plain_reasons(row)}\n{test['text'][i][:300]}\n")
```

The next cell runs the two agreement checks from section 4.6. DistilBERT splits words into pieces, so some disagreement with LIME (which uses whole words) is normal. Mention this in the report.

Notebook cell: agreement

```
from transformers import pipeline
from src.explain import lime_explain, lr_word_weights

clf = pipeline("text-classification", model=bert, tokenizer=bert_tok, top_k=None,
               device=0 if dev == "cuda" else -1)
shap_text = shap.Explainer(clf)

def top_shap(text, k=10):
    e = shap_text([text])[0, :, "AI"]
    order = np.argsort(-np.abs(e.values))[:k]
    return {str(e.data[i]).strip().lower() for i in order}

def top_lime(text, k=10):
    f = lambda t: predict_proba(t, bert, bert_tok, device=dev)
    return {w.lower() for w, _ in lime_explain(f, text, num_features=k)}

sample = test["text"].sample(20, random_state=SEED)
jac = []
for t in sample:
    a, b = top_shap(t), top_lime(t)
    jac.append(len(a & b) / len(a | b))
print("DistilBERT: SHAP vs LIME top-10 overlap (Jaccard):", round(float(np.mean(jac)), 2))

# LIME sanity check on the glass-box model, where the true word weights are known
hits = []
for t in sample:
    exact = lr_word_weights(lr, t)
    ranked = sorted(exact.items(), key=lambda kv: -abs(kv[1]))
    true_top = [w for w, _ in ranked if " " not in w][:10]        # single words only
    lime_top = [w.lower() for w, _ in lime_explain(lr.predict_proba, t, num_features=10)]
    hits.append(len(set(true_top) & set(lime_top)) / 10)
print("LIME recovers the true top words of LR:", round(float(np.mean(hits)), 2))

# coloured text view of one AI text (shows in the notebook)
shap.plots.text(shap_text([test["text"][cases["true_ai"]]])[0, :, "AI"])
```

7.10 Step 9: save the models and push

Notebook cell: save

```
import joblib

joblib.dump(xgb, "models/xgb.joblib")
joblib.dump(lr, "models/tfidf_lr.joblib")
with open("models/feature_names.json", "w") as f:
    json.dump(FEATS, f)
```

Notebook cell: push results to GitHub

```
!git add src app.py reports models/xgb.joblib models/tfidf_lr.joblib models/feature_names.json
!git commit -m "Trained models and figures"
!git push
```

> **Keep big files out of Git**
>
> The `.gitignore` in the repo already skips `data/` and `models/distilbert/`. The saved DistilBERT model is about 260 MB. Keep it in Google Drive or push it to the Hugging Face Hub with `bert.push_to_hub("your-hf-name/glassquill-distilbert")`. The app does not need it.

8. The web app

The app shows the explainable models only: XGBoost for the verdict with SHAP reasons and the TF-IDF model for word highlights. DistilBERT is left out to keep the app small and fast. All model logic sits in `src/detector.py`, so the Gradio and Streamlit front ends stay short.

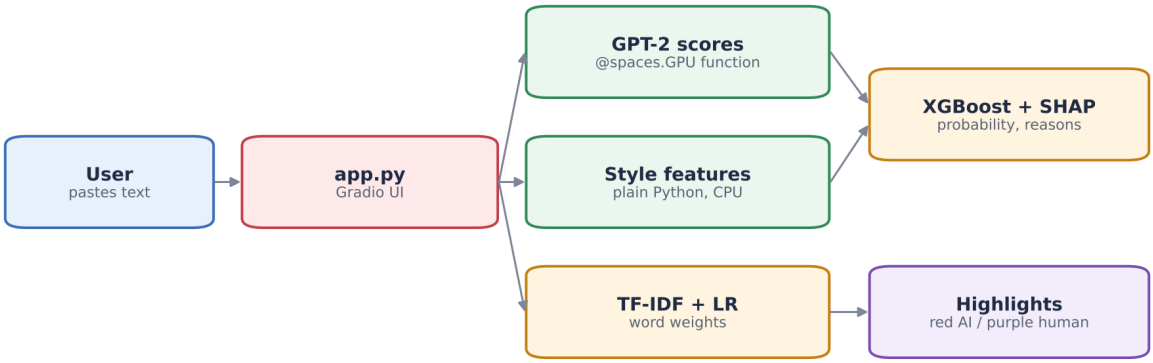


Figure 2. What happens when a user presses Analyse.

src/detector.py

```
"""Everything the app needs: load saved models once, then analyse a text."""
import json
import joblib
import pandas as pd
import shap

from src.explain import highlight, lr_word_weights, plain_reasons, shap_local_figure
from src.features import PerplexityScorer, stylometric_features


class Detector:
    def __init__(self, model_dir="models", device=None, lm_name="gpt2"):
        self.xgb = joblib.load(f"{model_dir}/xgb.joblib")
        self.lr = joblib.load(f"{model_dir}/tfidf_lr.joblib")
```

```
        with open(f"{model_dir}/feature_names.json") as f:
            self.feature_names = json.load(f)
        self.explainer = shap.TreeExplainer(self.xgb)
        self.scorer = PerplexityScorer(lm_name, device=device)

    def neural_features(self, text):
        """The only part that needs GPT-2 (and a GPU if one is there)."""
        return self.scorer.features(text)

    def analyze(self, text, neural=None):
        feats = stylometric_features(text)
        feats.update(neural if neural is not None else self.neural_features(text))
        x = pd.DataFrame([feats])[self.feature_names]
        p_ai = float(self.xgb.predict_proba(x)[0, 1])
        fig, sv_row = shap_local_figure(self.explainer, x)
        return {
            "p_ai": p_ai,
            "p_ai_lr": float(self.lr.predict_proba([text])[0, 1]),
            "shap_fig": fig,
            "reasons": plain_reasons(sv_row),
            "highlights": highlight(text, lr_word_weights(self.lr, text)),
        }
```

app.py

```
"""GlassQuill demo: paste text, get a verdict and the reasons behind it."""
import os
import gradio as gr
import matplotlib.pyplot as plt

try:                                    # present on Hugging Face Spaces
    import spaces
    gpu = spaces.GPU
except ImportError:                     # laptop or Colab: do nothing
    def gpu(*args, **kwargs):
        return args[0] if args and callable(args[0]) else (lambda f: f)

from src.detector import Detector

DEVICE = "cuda" if os.getenv("SPACES_ZERO_GPU") else None   # None = auto
det = Detector("models", device=DEVICE)


@gpu(duration=15)                       # seconds of GPU time asked for per call
def neural_features(text):
    return det.neural_features(text)


def run(text):
    if len(text.split()) < 50:
        raise gr.Error("Please paste at least 50 words.")
    plt.close("all")
    r = det.analyze(text, neural=neural_features(text))
    verdict = {"AI-generated": r["p_ai"], "Human-written": 1 - r["p_ai"]}
    reasons = (f"**Main model (XGBoost) says {r['p_ai']:.0%} AI.** "
               f"Word model (TF-IDF + LR) says {r['p_ai_lr']:.0%} AI.\n\n"
               f"Top reasons:\n{r['reasons']}")
    return verdict, reasons, r["shap_fig"], r["highlights"]


with gr.Blocks(title="GlassQuill") as demo:
    gr.Markdown("# GlassQuill\nExplainable detection of AI-generated text. "
                "This is a study project. Do not use it alone to accuse anyone.")
    with gr.Row():
        with gr.Column():
            text = gr.Textbox(lines=12, label="Paste text (50 to 400 words works best)")
            btn = gr.Button("Analyse", variant="primary")
        with gr.Column():
            label = gr.Label(label="Verdict", num_top_classes=2)
            reasons = gr.Markdown()
    plot = gr.Plot(label="Why: SHAP contribution of each feature")
    words = gr.HighlightedText(label="Words that pushed towards AI (red) or human (purple)")
    btn.click(run, inputs=text, outputs=[label, reasons, plot, words])
```

```
if __name__ == "__main__":
    demo.launch()
```

The `gpu` decorator is the only Hugging Face specific part. On a ZeroGPU Space it asks for a GPU for at most 15 seconds while GPT-2 runs. On a laptop or in Colab it does nothing. In the highlighted text, positive scores show in red (towards AI) and negative scores in purple (towards human).

8.1 Try it in Colab first

Notebook cell: run the app

```
from app import demo
demo.launch(share=True)     # prints a temporary public link
```

Test it with one paragraph you wrote yourself and one paragraph from ChatGPT. Take screenshots for the report.

9. Deployment

> **Hugging Face changed its free tier**
>
> The Hugging Face docs now say that Gradio and Docker Spaces need a paid plan, with one exception: free personal accounts in good standing (verified email, account older than 30 days) can host up to **two Gradio Spaces on ZeroGPU**. ZeroGPU lends a slice of a shared GPU only while a function marked with `@spaces.GPU` runs. Visitors get a small daily GPU allowance (about 2 minutes without logging in, 5 minutes with a free account). Each analysis here needs only a few seconds, so this is enough for a demo.

9.1 Plan A: Hugging Face Space on ZeroGPU

1. Finish step 9 so that `models/` has `xgb.joblib`, `tfidf_lr.joblib` and `feature_names.json`.
2. Log in and create the Space. On huggingface.co you can also do this from **New Space** with SDK Gradio and hardware ZeroGPU.
3. Add the settings block below to the top of the Space's `README.md`.
4. Upload the code and the small model files. Leave out data, the notebook and DistilBERT.
5. Open the Space page and watch the **Logs** tab. The first build takes a few minutes.
6. Test it with one human and one AI paragraph, then put the link in the report and the slides.

Terminal (laptop or a Colab cell starting with !)

```
pip install -U huggingface_hub
hf auth login                       # paste a token with write access
hf repos create glassquill --type space --sdk gradio --flavor zero-a10g
hf upload <your-hf-name>/glassquill . --repo-type space \
    --exclude "data/*" --exclude "models/distilbert/*" --exclude "notebook*" --exclude "docs/*"
```

Top of the Space README.md

```
---
title: GlassQuill
colorFrom: indigo
colorTo: gray
sdk: gradio
sdk_version: 6.30.0
python_version: "3.12"
app_file: app.py
pinned: false
---
```

A free Space goes to sleep when nobody uses it. The first visit wakes it up, which can take a minute. Open it ten minutes before you present.

9.2 Plan B: Colab share link (always works)

Run the "run the app" cell from section 8. Gradio prints a public link that works while the Colab session is running. Keep this ready on demo day in case the Space is down.

9.3 Plan C: Streamlit Community Cloud

Streamlit Community Cloud is free and deploys straight from a GitHub repo. Its docs list a memory limit of about 2.7 GB, which is enough for GPT-2 small on CPU. Connect the glassquill repo, choose `streamlit_app.py` as the main file and add `streamlit` to the requirements. To keep the install small, add the line `--extra-index-url https://download.pytorch.org/whl/cpu` at the top of the requirements file so pip picks the CPU build of torch.

streamlit_app.py

```
"""Plan C front end for Streamlit Community Cloud. Same Detector, CPU only."""
import streamlit as st
from src.detector import Detector

st.set_page_config(page_title="GlassQuill")


@st.cache_resource                      # load the models once, not on every click
def load_detector():
    return Detector("models", device="cpu")


st.title("GlassQuill")
st.caption("Explainable detection of AI-generated text. Study project only.")
text = st.text_area("Paste text (50 to 400 words works best)", height=250)

if st.button("Analyse", type="primary"):
    if len(text.split()) < 50:
        st.error("Please paste at least 50 words.")
    else:
        r = load_detector().analyze(text)
        st.metric("Chance it is AI-generated (XGBoost)", f"{r['p_ai']:.0%}")
        st.caption(f"Word model (TF-IDF + LR): {r['p_ai_lr']:.0%} AI")
        st.markdown("**Top reasons**\n\n" + r["reasons"])
        st.pyplot(r["shap_fig"])
```

9.4 Deployment checklist

Check	Done
Model files in `models/` and loading without errors	☐
App rejects texts under 50 words with a clear message	☐
One human and one AI example give sensible verdicts	☐
SHAP plot and word highlights show up	☐
Disclaimer is visible on the page	☐
Space link, Colab backup and screenshots ready for the presentation	☐

10. How this maps to the ESE rubric

Component	Marks	What we deliver	Where it shows
Problem refinement and use case relevance	3	Problem statement, RQ1 to RQ4, stakeholders, scope, bias case	Report ch. 1, slides 2 to 3
Dataset understanding, preprocessing, features	4	HC3 and MAGE described, quirks found and fixed, 18 features, grouped split and grouped CV	Report ch. 3, EDA plots
Methodology and ML pipeline design	5	Four-model pipeline with reasons, tuning plan, threshold rule, evaluation and explanation plan	Figure 1, model table
Implementation and project execution	5	Modular repo, fixed seeds, saved artifacts, working app with public link	GitHub repo, live demo
Results, metrics and comparison	5	Metrics on three test sets, FPR focus, confusion matrices, ROC curves, SHAP and LIME analysis	Results tables, figures
Final report quality	3	Report structure in section 12 with IEEE references	Report PDF
Communication and presentation	2	Twelve slides and a live demo with a backup	Slides
Viva and project defence	3	Both members prepared with section 13	Viva

11. Plan and work split

A four-week plan. Squeeze or stretch it to fit the submission date.

Week	Work	Done when
1	Accounts, repo, Colab setup, read the background, steps 2 and 3, EDA	Splits saved and EDA plots made
2	Steps 4 to 6: features and all four models, in-domain results	Results table for the HC3 test set
3	Step 7 robustness tests, step 8 explanations, app running in Colab	All tables and figures for the report
4	Deployment, report, slides, viva practice	Report submitted, demo link working

Person	Owns	Report chapters
A	Data loading and cleaning, features, logistic regression, XGBoost, SHAP analysis	3, 4 and the SHAP part of 7
B	DistilBERT, MAGE and paraphrase tests, LIME and agreement checks, app and deployment	5, 6 and the LIME part of 7
Both	Problem statement, results discussion, slides, viva practice	1, 2, 8 and 9

The viva is individual

Each person must be able to explain every part of the project, not just their own half. Walk each other through your code before week 4.

12. Report and presentation

12.1 Report outline

Section	What goes in
Abstract	150 to 200 words: problem, method, main numbers, one key finding
1 Introduction	Problem, motivation (bias case), research questions, stakeholders, scope
2 Literature review	GLTR, DetectGPT, Fast-DetectGPT and Binoculars, commercial detectors, HC3, MAGE, RAID, Liang et al., SHAP and LIME
3 Dataset	Sources, quirks found, cleaning steps, split, EDA plots
4 Methodology	Figure 1, feature table, the four models, tuning, evaluation and explanation plan
5 Implementation	Tools, repo layout, app screenshots, deployment
6 Results	Tables for the three test sets, confusion matrices, ROC curves, per-source MAGE errors
7 Explainability analysis	Global SHAP, four local cases, LR top words, the two agreement checks
8 Limitations and ethics	False positives, bias, paraphrasing, drift, English only
9 Conclusion and future scope	Answers to RQ1 to RQ4, then future work
References	IEEE style, see section 15
Appendix	Repo and app links, extra plots

12.2 Future scope ideas

- Detectors for Hindi, Marathi and other Indian languages.
- Newer zero-shot scores such as Fast-DetectGPT or Binoculars as extra features.
- Sentence-level detection for documents that mix human and AI writing.
- Watermark-based detection.
- Choosing the threshold for a fixed low false positive rate, for example 1%.

12.3 Slide plan (12 slides)

#	Slide	#	Slide
1	Title, team, guide	7	The four models
2	Problem and the bias case	8	In-domain results table
3	Objective and research questions	9	MAGE and paraphrase results
4	Dataset and cleaning	10	Explanations: SHAP beeswarm and one waterfall
5	Pipeline (Figure 1)	11	Live demo
6	Features with the expected direction	12	Limitations, future scope, thank you

13. Viva preparation

Q. What is perplexity and why is it lower for AI text?

It is the exponential of the average negative log probability of each token under a language model. LLMs pick likely tokens, so another language model finds their text easy to predict.

Q. Why GPT-2 and not a newer model to compute perplexity?

It is small, free, fast on a T4 and well studied. A newer model may score better but costs more. Swapping it is listed as future work.

Q. How are SHAP values calculated?

They are Shapley values from game theory: the average change in the prediction when a feature is added, over all orders of features. TreeSHAP computes them exactly for tree models.

Q. How is SHAP different from LIME?

SHAP has a solid theory and its values add up to the prediction. LIME fits a small local linear model on random changes to the input, so it is faster for any model but approximate and can change between runs.

Q. Why did you split by question?

HC3 has human and AI answers to the same question. Without grouping the model could learn topics instead of style and the test score would be too high.

Q. Why focus on the false positive rate?

A false positive accuses a human of using AI. That error hurts real people, so it matters more than a missed AI text.

Q. Why is accuracy not enough?

It hides which errors you make. Two models with the same accuracy can have very different false positive rates.

Q. What does XGBoost's best feature tell you?

Answer from your own SHAP bar plot. Explain the direction too, for example "low log perplexity pushes towards AI".

Q. Why does performance drop on MAGE?

MAGE has other LLMs, other domains and other writing styles. The models learned ChatGPT on HC3 topics, so some of what they learned does not carry over.

Q. How can someone fool your detector?

Paraphrasing, mixing human and AI sentences, asking the LLM for a casual style or adding typos. Our paraphrase test measures the first one.

Q. Is your detector fair to non-native writers?

Not proven. Simple wording has low perplexity, so non-native writers risk being flagged. That is why the app shows reasons and a disclaimer.

Q. What would you do with more time?

Test on RAID, add newer zero-shot scores, cover Indian languages and set the threshold for a fixed low false positive rate.

14. Troubleshooting

Problem	Cause	Fix
`RuntimeError: Dataset scripts are no longer supported, but found HC3.py`	The datasets library dropped loading scripts in 4.0	Use `load_hc3()`, which reads `all.jsonl` with pandas
`LookupError: Resource cmudict not found`	textstat needs an NLTK word list	`features.py` downloads it. Or run `nltk.download("cmudict")`
`ImportError: ... requires accelerate`	Trainer needs accelerate	`pip install accelerate`
CUDA out of memory	Batch too big for the GPU	Batch size 8, or `max_length=128`
Test accuracy above 99%	A shortcut or leakage	Check cleaning, the group split, length histograms and the top TF-IDF words
ROC-AUC on MAGE below 0.5	MAGE labels not flipped	`ood["label"] = 1 - ood["label"]`
Space build fails	Missing package or wrong settings	Read the Logs tab. Check `requirements.txt` and the README block
Space says no GPU function was found	ZeroGPU needs at least one `@spaces.GPU` function	Keep the `gpu` decorator in `app.py`
Cannot pick ZeroGPU or asked to upgrade	Account newer than 30 days or email not verified	Use plan B or plan C
Colab cannot clone the repo	Private repo without a token	Add `GITHUB_TOKEN` in Colab Secrets (step 2)
Colab disconnects during a long cell	Free tier time limits	Each step saves parquet files. Load them instead of recomputing

15. References

[1] B. Guo et al., "How close is ChatGPT to human experts? Comparison corpus, evaluation, and detection," arXiv:2301.07597, 2023.

[2] Y. Li et al., "MAGE: Machine-generated text detection in the wild," arXiv:2305.13242, 2024.

[3] L. Dugan et al., "RAID: A shared benchmark for robust evaluation of machine-generated text detectors," in Proc. ACL, 2024, pp. 12463-12492.

[4] W. Liang, M. Yuksekgonul, Y. Mao, E. Wu and J. Zou, "GPT detectors are biased against non-native English writers," Patterns, vol. 4, no. 7, 2023.

[5] S. Gehrmann, H. Strobelt and A. M. Rush, "GLTR: Statistical detection and visualization of generated text," in Proc. ACL System Demonstrations, 2019.

[6] E. Mitchell, Y. Lee, A. Khazatsky, C. D. Manning and C. Finn, "DetectGPT: Zero-shot machine-generated text detection using probability curvature," in Proc. ICML, 2023.

[7] G. Bao, Y. Zhao, Z. Teng, L. Yang and Y. Zhang, "Fast-DetectGPT: Efficient zero-shot detection of machine-generated text via conditional probability curvature," in Proc. ICLR, 2024.

[8] A. Hans et al., "Spotting LLMs with Binoculars: Zero-shot detection of machine-generated text," in Proc. ICML, 2024.

[9] J. Kirchenbauer et al., "A watermark for large language models," in Proc. ICML, 2023.

[10] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Proc. NeurIPS, 2017.

[11] S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees," Nature Machine Intelligence, vol. 2, pp. 56-67, 2020.

[12] M. T. Ribeiro, S. Singh and C. Guestrin, "Why should I trust you? Explaining the predictions of any classifier," in Proc. ACM SIGKDD, 2016.

[13] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. ACM SIGKDD, 2016.

[14] V. Sanh, L. Debut, J. Chaumond and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," arXiv:1910.01108, 2019.

[15] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI, 2019.

[16] Hugging Face, "Spaces overview" and "ZeroGPU" documentation, huggingface.co/docs/hub, accessed October 2026.